

# Code Explanation

## Testing Aggregate Module

The given code is a test suite for the aggregate module, which is part of a larger data processing pipeline. The aggregate module is responsible for creating various analytics DataFrames from enriched sales data.

The test suite is written using the Pytest framework and utilizes the PySpark library for data processing. It tests four main functions: `create_sales_summary`, `create_product_analytics`, `create_customer_analytics`, and `create_time_series_analytics`. These functions are responsible for generating different types of analytics DataFrames.

### ## Overview of the Code

The code defines a test suite for the aggregate module. It includes a fixture to create an enriched sales DataFrame and four test functions to verify the correctness of the analytics DataFrames generated by the aggregate module.

#### ## Step 1: Defining the Test Fixture

The test fixture `enriched_sales_df` is used to create an enriched sales DataFrame for testing purposes. This fixture is created by applying a series of transformations to the sample sales data.

```
@pytest.fixture
def enriched_sales_df(sample_sales_data, sample_products_data, sample_customers_data):
    transformed_df = clean_and_transform_data(sample_sales_data)
    enriched_df = enrich_sales_data(transformed_df, sample_products_data, sample_customers_data)
    return add_sales_metrics(enriched_df)
```

#### ## Step 2: Testing the `create_sales_summary` Function

The `test_create_sales_summary` function tests the `create_sales_summary` function from the aggregate module. It verifies that the resulting DataFrame has the expected columns and that the aggregation logic is correct.

```
def test_create_sales_summary(enriched_sales_df):
    result_df = create_sales_summary(enriched_sales_df)
    assert isinstance(result_df, DataFrame)
    assert "total_quantity" in result_df.columns
    assert "total_sales" in result_df.columns
    assert "transaction_count" in result_df.columns
    assert "unique_customers" in result_df.columns
    assert "average_price" in result_df.columns
```

#### ## Step 3: Testing the `create_product_analytics` Function

The `test_create_product_analytics` function tests the `create_product_analytics` function from the aggregate module. It checks that the resulting DataFrame has the expected columns and that the number of products is correct.

```
def test_create_product_analytics(enriched_sales_df):
    result_df = create_product_analytics(enriched_sales_df)
    assert isinstance(result_df, DataFrame)
    assert "total_quantity_sold" in result_df.columns
    assert "total_revenue" in result_df.columns
    assert "average_price" in result_df.columns
    assert "unique_customers" in result_df.columns
    assert "days_sold" in result_df.columns
    assert "revenue_per_day" in result_df.columns
    assert result_df.count() == 3
```

## Step 4: Testing the `create\_customer\_analytics` Function

The `test\_create\_customer\_analytics` function tests the `create\_customer\_analytics` function from the aggregate module. It verifies that the resulting DataFrame has the expected columns.

```
def test_create_customer_analytics(enriched_sales_df):
    result_df = create_customer_analytics(enriched_sales_df)
    assert isinstance(result_df, DataFrame)
    assert "total_spend" in result_df.columns
    assert "transaction_count" in result_df.columns
    assert "shopping_days" in result_df.columns
    assert "unique_products" in result_df.columns
    assert "last_purchase_date" in result_df.columns
    assert "average transaction value" in result_df.columns
```

## Step 5: Testing the `create\_time\_series\_analytics` Function

The `test\_create\_time\_series\_analytics` function tests the `create\_time\_series\_analytics` function from the aggregate module. It checks that the resulting DataFrame has the expected columns.

```
def test_create_time_series_analytics(enriched_sales_df):
    result_df = create_time_series_analytics(enriched_sales_df)
    assert isinstance(result_df, DataFrame)
    assert "daily_sales" in result_df.columns
    assert "transaction_count" in result_df.columns
    assert "unique_customers" in result_df.columns
    assert "average basket size" in result_df.columns
```